AI Review Bootcamp

Day 8 Lab: AI Review, CI/CD, Docker, and Jenkins

1 Introduction

Modern software engineering increasingly relies on AI-assisted development. Large Language Models (LLMs) can rapidly generate source code, test cases, documentation, and debugging suggestions. However, AI-generated software artifacts may still contain logical errors, hallucinated APIs, hidden edge-case failures, and inconsistent subsystem behaviour.

Therefore, AI-assisted software engineering still requires:

- systematic verification
- automated regression testing
- subsystem-level review
- cross-tool checking
- CI/CD automation

This lab introduces a simplified but realistic workflow combining:

- AI-assisted development
- AI review methodologies
- Docker-based deployment
- Jenkins-based CI/CD
- Python verification scripts
- C++ implementation modules

The lab contains two questions:

- Question 1: Guided AI Review + CI/CD Hello-World Exercise
- Question 2: AI-Assisted Pathfinding System

2 AI Review Concepts Used in This Lab

The lecture covers the main AI Review concepts. This lab applies those concepts through a small CI/CD workflow.

In this lab, students will practise the following AI Review methods:

- **AI-generated test expansion**: use AI to propose additional test cases and corner cases.
- **Cross-AI verification**: use another AI tool, or a different prompt, to review AI-generated code or tests.
- **Divide-and-conquer review**: review the C++ implementation, Python verification script, Flask interface, and Jenkins workflow separately.
- **Regression verification**: use Jenkins to check whether code changes break existing behaviour.
- **Failure diagnosis**: use Jenkins Console Output and AI assistance to identify why a build or test failed.

The key message is:

AI can help generate software quickly, but correctness must still be verified.

3 Required Technologies, Installation, and Verification

This lab uses Docker, Docker Compose, Jenkins, Git, Python, and C++.

3.1 Required Tools

Students should prepare:

- Docker Desktop
- Git
- Modern web browser

3.2 Installing Docker Desktop

Install Docker Desktop using the official installer for Mac or Windows.

After installation:

1. Start Docker Desktop
2. Wait until Docker Engine is running
3. Open a terminal
4. Verify installation

Verify Docker:

```
docker --version
docker compose version
```

3.3 Installing Jenkins

Students do not install Jenkins directly.

Jenkins runs inside a Docker container using:

```
Dockerfile.jenkins
```

The Jenkins web interface will later be available at:

```
http://localhost:60002
```

3.4 Installing and Verifying Git

Verify Git:

```
git --version
```

3.5 Important Docker Concepts

- Image: packaged software template
- Container: running instance of an image
- Volume: persistent storage
- Port mapping: mapping container ports to host ports
- Docker Compose: manages multiple containers

3.6 Important Docker Commands

Start containers:

```
docker compose up --build -d
```

Meaning:

- `up`: create and start services
- `--build`: rebuild images
- `-d`: detached mode (background execution)

Check running containers:

```
docker compose ps
```

View logs:

```
docker compose logs
```

Follow Jenkins logs:

```
docker compose logs -f jenkins
```

Run command inside Jenkins container:

```
docker compose exec jenkins python3 --version
```

Stop containers:

```
docker compose down
```

3.7 Port Mapping

Flask mapping:

```
60001:5000
```

Meaning:

- 5000: internal Flask container port
- 60001: host browser-accessible port

Therefore Flask is accessed using:

```
http://localhost:60001
```

Jenkins mapping:

```
60002:8080
```

Therefore Jenkins is accessed using:

```
http://localhost:60002
```

4 Environment Setup

Extract:

```
Day_8_Q1.zip
```

Expected structure:

```
Day_8_Q1/

  cpp/
    multiply.cpp

  python/
    check_multiply.py

  Dockerfile
  Dockerfile.jenkins
  docker-compose.yml
  requirements.txt
  app_multiply.py
```

Verify Docker installation:

```
docker --version
docker compose version
```

5 Question 1 – Guided AI Review and CI/CD Exercise

5.1 Objective

The objective of this exercise is to:

- understand Docker Compose workflow
- understand Jenkins CI/CD pipeline execution
- understand AI review methodologies
- understand automated verification
- observe regression detection

5.2 Exercise Phases

This exercise contains:

1. Docker Compose + Flask application
2. Jenkins container setup
3. Jenkins runs Python tests
4. Jenkins compiles and runs C++
5. Jenkins runs integrated Python + C++ workflow
6. AI review tasks

5.3 Starting Docker Compose

Open a terminal in the project directory:

```
docker compose up --build -d
```

Verify running containers:

```
docker compose ps
```

5.4 Flask Web Application

Open:

```
http://localhost:60001
```

The Flask application:

- calls the C++ executable
- obtains multiplication results
- combines results using Python
- displays results in browser

5.5 Accessing Jenkins

Open:

```
http://localhost:60002
```

Open Jenkins job:

```
phase3-python-test
```

5.6 Jenkins Workflow

Select:

```
Build Now
```

The Jenkins workflow performs:

1. compile C++ source code
2. generate executable binary
3. run Python verification script
4. compare expected vs actual outputs
5. generate PASS / FAIL result

Troubleshooting note: executable path
The Python verification script must call the compiled C++ executable from the correct working directory. In this lab, Jenkins should run from the project root inside the container, normally `/workspace`.
If the verification script cannot find `./multiply`, check the Jenkins working directory in the Console Output. In some environments, the script may need to call the executable using the absolute container path:

```
["/workspace/multiply", str(a), str(b)]
```

This is a path / working-directory issue, not a multiplication-logic issue.

5.7 Viewing Console Output

Students should inspect:

```
Build History -> Console Output
```

Observe:

- C++ compilation
- Python verification
- SUCCESS / FAILURE reporting

5.8 AI Review Activities

Students must complete the following AI review activities:

1. Use an AI tool to propose additional test cases for `multiply.cpp`.
2. Add at least four new test cases into `python/check_multiply.py`.
3. Run Jenkins and confirm that all tests pass.
4. Intentionally introduce a bug into `multiply.cpp`.
5. Run Jenkins again and capture the failure.
6. Paste the Jenkins Console Output into an AI tool and ask it to diagnose the failure.
7. Use another AI tool, or a different prompt, to review the C++ implementation and Python verification script.
8. Write a short comparison of the AI review findings.

5.9 AI Review Task A – AI-Generated Test Cases

Use an AI tool to generate additional corner cases for:

```
multiply.cpp
```

Examples:

- negative numbers
- zero
- large values
- invalid input

Add these cases into:

```
python/check_multiply.py
```

Re-run Jenkins verification.

5.10 AI Review Task B – Cross-AI Verification

Use another AI tool to review the generated code.
Compare:

- correctness
- edge-case handling
- coding style
- maintainability

5.11 AI Review Task C – Intentional AI Failure

Use AI to intentionally generate incorrect logic.

Example:

Replace:

```
a * b
```

with:

```
a + b
```

Then:

1. rebuild Jenkins job
2. observe FAILURE
3. inspect Console Output
4. restore correct implementation

5.12 AI Review Reflection

Students should explain:

- why AI-generated code may still fail
- why automated verification is important
- strengths and weaknesses of different AI tools
- why subsystem decomposition is useful

6 Question 2 – AI-Assisted Pathfinding System

6.1 Objective

In this question, you will build a small pathfinding and map visualisation system. The goal is not only to generate working code with AI, but also to practise AI review, test design, and verification.

Your system should:

1. load map data from text files;
2. compute the shortest path between two selected nodes;
3. visualise the map and highlight the shortest path;
4. provide a simple web interface for pathfinding;
5. protect map-editing operations using a simple authentication mechanism;
6. use AI review methods to test and improve the design.

The pathfinding function should be available to all users. However, changing the map data should be allowed only for authorised users.

6.2 System Overview

The recommended system structure is shown below.

Component	Purpose
`Node_Info.txt`	Defines nodes, coordinates, location type, and name
`Graph_Path.txt`	Defines paths between nodes and their distances
Pathfinding algorithm	Computes the shortest path using distance as the cost
Map visualisation	Draws nodes, paths, building types, and highlighted route
Web interface	Allows users to select start/end nodes and view results
Editor authentication	Protects add/edit/delete operations for map data
Verification script	Tests expected pathfinding behaviours

The web application should run at:

`http://localhost:60003`

This fixed port allows supporting engineers and instructors to test student solutions consistently. In production systems, public web services normally use standard HTTP/HTTPS ports or a reverse proxy. In this lab, using `localhost:60003` is sufficient.

6.3 Map Data Files

The map is represented by two text files:

- `Node_Info.txt`
- `Graph_Path.txt`

Both files may contain comment lines. A comment line starts with the character `#`. Your program should ignore empty lines and comment lines.

6.3.1 Node Information File

The file `Node_Info.txt` defines all nodes on the map.

Each non-comment line has the following format:

```
NodeID X Y TypeID Name
```

Example:

```
# NodeID X Y TypeID Name
0 10 10 0 School_A
1 25 12 1 Shop_A
2 40 20 2 Mall_A
3 55 30 3 HDB_A
4 70 25 4 Park_A
```

The fields are defined as follows:

Field	Meaning
`NodeID`	Unique integer ID of the node
`X`	X-coordinate used for plotting
`Y`	Y-coordinate used for plotting
`TypeID`	Location or building type
`Name`	Human-readable node name without spaces

The `TypeID` should be interpreted as follows:

`TypeID`	Location type	Suggested visual style
0	School	Wide green rectangle
1	Shop	Thin yellow rectangle
2	Mall	Tall blue rectangle or large block
3	HDB / residential	Grey rectangle
4	Park / open area	Light green circle or rounded shape

The exact artistic style is not important. The important requirement is that different location types should be visually distinguishable.

6.3.2 Graph Path File

The file `Graph_Path.txt` defines all paths between nodes.

Each non-comment line has the following format:

```
EdgeID NodeA NodeB Distance
```

Example:

```
# EdgeID NodeA NodeB Distance
0 0 1 120
1 1 2 180
2 0 3 250
3 3 4 160
4 4 2 100
```

The fields are defined as follows:

Field	Meaning
`EdgeID`	Unique integer ID of the path
`NodeA`	Node ID at one end of the path
`NodeB`	Node ID at the other end of the path
`Distance`	Travel distance or cost of this path

All paths can be treated as undirected unless you explicitly document another choice.

The value of `Distance` is the path cost used by the shortest-path algorithm. It may be different from the direct straight-line distance between the two node coordinates. For example, a walking path may curve around a building or follow a road.

6.4 Functional Requirements

Your system should satisfy the following requirements.

6.4.1 Pathfinding

The system should allow a user to select or enter:

- a start node ID;
- an end node ID.

The system should compute the shortest path using the `Distance` value from `Graph_Path.txt`.
The output should include:

- the shortest path as a list of node IDs;
- the total distance;
- a clear message if no path exists;
- a clear message if the start or end node ID is invalid.

Example output:

```
Start node: 0
End node: 2

Shortest path:
0 -> 1 -> 2

Total distance:
300
```

You may use Dijkstra's algorithm or another correct shortest-path algorithm for graphs with non-negative edge distances.

6.4.2 Map Visualisation

The system should plot the map using coordinates from `Node_Info.txt`.

Coordinate convention. The coordinates in `Node_Info.txt` use a map-style coordinate system: `X` increases to the right and `Y` increases upward.

Some graphics libraries use screen coordinates, where `Y` increases downward. If your visualisation uses such a library, you may convert the coordinates internally before drawing. The final map should preserve the relative positions of all nodes.

The origin location does not need to be shown explicitly. The visualisation should scale and translate the map automatically based on all node coordinates.

The visualisation should show:

- all nodes;
- all paths;
- labels or IDs for nodes;
- different visual styles for different `TypeID` values;
- the selected shortest path highlighted clearly.

6.4.3 Web Interface

The pathfinding web application should be available at:

`http://localhost:60003`

The web interface should allow public users to:

- view the map;
- select or enter start and end nodes;
- run pathfinding;
- view the result path and total distance.

6.4.4 Map Editing and Authentication

The pathfinding service should be public, but map editing should be protected.

Public users should be able to:

- view the map;
- request a shortest path.

Only authorised users should be able to:

- add a node;
- edit a node;
- delete a node;
- add a path;
- edit a path;
- delete a path;
- save updated map data.

For this lab, a simple authentication mechanism is sufficient. Acceptable examples include:

- a fixed editor password;
- a simple login form;
- an editor token stored in a configuration file;
- a protected edit API that requires a shared token.

You are not required to implement a full production security system. The main requirement is to separate public pathfinding from protected map editing.

6.5 AI Review Requirements

This question is part of the AI Review lab. You should not only ask AI to generate code. You should also ask AI to review and test the system.

6.5.1 AI Review Task 1 – Review the Data Format

Ask an AI tool to review your proposed `Node_Info.txt` and `Graph_Path.txt` formats.

Suggested prompt:

```
Review this map data format for a pathfinding system.
Check whether the node file and edge file contain enough information
for shortest-path computation and map visualisation.
Identify missing assumptions, ambiguous fields, and edge cases.
```

Your review notes should discuss:

- whether the file format is clear;
- whether comment lines are handled;
- whether invalid node IDs can be detected;
- whether duplicate node IDs or edge IDs are handled;
- whether the distance field is clearly defined.

6.5.2 AI Review Task 2 – Generate Test Maps

Ask AI to generate additional test maps. A test map means an alternative `Node_Info.txt` and `Graph_Path.txt` pair.

The purpose is to check whether your program is data-driven and does not depend on one fixed map.

Examples of useful test maps include:

- a very small connected map;
- a map with multiple valid paths;
- a map with a disconnected component;
- a map with equal-distance shortest paths;
- a map where edge distance is not equal to direct coordinate distance;
- a map with different building types.

Suggested prompt:

```
Generate three small test maps for my pathfinding system.
Each map should include Node_Info.txt and Graph_Path.txt content.
Include comments starting with #.
Create one normal connected graph, one graph with a disconnected component,
and one graph with two equal shortest paths.
```

6.5.3 AI Review Task 3 – Generate Corner-Case Requests

Ask AI to generate corner-case pathfinding requests. A request means a start node and an end node, and the expected behaviour.

Examples:

- start node equals end node;
- start node does not exist;
- end node does not exist;
- no path exists;
- two or more paths have the same shortest distance;
- map contains a cycle;
- graph contains an isolated node.

Suggested prompt:

```
Given this Node_Info.txt and Graph_Path.txt, generate corner-case
pathfinding requests. For each request, provide the start node, end node,
expected behaviour, and why this case is important.
```

6.5.4 AI Review Task 4 – Review Authentication

Ask AI to review whether map editing is properly protected.

Suggested prompt:

```
Review this pathfinding web application for access-control mistakes.
Public users should be able to view the map and run pathfinding.
Only authorised users should be able to add, edit, delete, or save map data.
Identify routes, API calls, or UI actions that may accidentally allow
unauthorised map editing.
```

Your review should check:

- whether all edit actions require authentication;
- whether public pathfinding still works without login;
- whether the editor password or token is exposed unnecessarily;
- whether the system handles failed authentication clearly.

6.5.5 AI Review Task 5 – Review Tests and CI Evidence

Ask AI to review your test cases and CI output.

Suggested prompt:

```
Review these pathfinding tests and CI logs.
Do the tests check real pathfinding behaviour, or do they only execute code?
Identify missing test cases, weak expected outputs, and possible false confidence.
```

Your review should discuss:

- which behaviours are covered by your tests;
- which behaviours are not yet covered;
- whether the expected outputs are reliable;
- whether CI provides meaningful verification evidence.

6.6 Verification Requirements

Your solution should include a verification script or automated test procedure.

At minimum, test the following cases:

1. normal shortest path;
2. start node equals end node;
3. invalid start node;
4. invalid end node;
5. disconnected graph or unreachable destination;
6. map with multiple possible paths;
7. map-editing route rejected for unauthorised user;
8. public pathfinding still works without authentication.

For each test case, document:

- test input;
- expected output or expected behaviour;
- actual output;
- PASS or FAIL result.

If you use Jenkins, your CI job should compile or start the required components, run the verification script, and show a clear SUCCESS or FAILURE result.

6.7 Suggested Project Directory

A possible project structure is:

```
pathfinding_project/
app.py
requirements.txt
docker-compose.yml
data/
Node_Info.txt
Graph_Path.txt
src/
pathfinder.py
map_loader.py
auth.py
tests/
test_pathfinder.py
test_auth.py
docs/
ai_review_notes.md
test_plan.md
ci_result_summary.md
```

You may use a different structure, but your files should be organised clearly.

6.8 Deliverables for Question 2

Submit the following:

1. source code for the pathfinding system;
2. `Node_Info.txt` and `Graph_Path.txt`;
3. screenshots of the web interface and map visualisation;
4. screenshot or log showing pathfinding result;
5. screenshot or log showing authentication for map editing;
6. test cases and expected outputs;
7. Jenkins or command-line verification output;
8. AI review notes, including:
    - data-format review;
    - AI-generated test maps;
    - AI-generated corner-case requests;
    - authentication review;
    - test and CI review.

6.9 Assessment Focus

This question is assessed mainly on engineering correctness and AI review quality, not visual beauty. Important assessment points include:

- correct parsing of `Node_Info.txt` and `Graph_Path.txt`;
- correct shortest-path computation;
- clear visualisation of nodes, edges, building types, and selected route;
- clear separation between public pathfinding and protected map editing;
- meaningful test cases and expected outputs;
- evidence from CI or repeatable command-line verification;
- useful AI review notes and human judgement.

6.10 Important Notes

- Do not hardcode one specific map into the algorithm.
- Your program should work when the map files are changed.
- The `Distance` value in `Graph_Path.txt` is the pathfinding cost.
- The coordinate values in `Node_Info.txt` are for plotting.
- The shortest path should be based on `Distance`, not straight-line coordinate distance.
- AI-generated code must be reviewed and tested before being trusted.

7 Deliverables

Students should submit:

- modified source code
- screenshots of Flask application
- screenshots of Jenkins SUCCESS
- screenshots of Jenkins FAILURE
- AI-generated prompts used
- AI-generated test cases
- short reflection on AI review findings
- README execution guide

8 Optional Advanced Extensions

Students may optionally explore:

- Git-triggered CI/CD
- scheduled periodic Jenkins builds
- REST APIs
- AI-generated pipeline scripts
- multi-agent review workflows